

Stock Buy and Sell Problem: Variant 1

Maximizing Profit with a Single Transaction

Introduction

Welcome to DSA 1! Today, we delve into a popular array problem: **Stocks Buy and Sell**. We'll focus on the first variant where you are allowed to buy and sell a stock only **once**. Other variants (e.g., at most 2 transactions, at most k transactions, multiple transactions) will be discussed in future lessons.

Problem Statement

You are given an array of stock prices, where `prices[i]` is the price of a given stock on day `i`.

```
^ Prices
|
|  _ / \ _
| /   / \
+-----> Days
Your task is to:
```

- Buy **one** stock and sell it **once**.
- Maximize your profit.
- Return the **maximum profit** you can achieve.

Key Assumption: You know the future prices of the stock. This is not how it works in the real world, but for this problem, it simplifies the task to finding optimal points.

Constraint: You must buy the stock before you sell it. You cannot sell a stock before buying it.

Example: If prices are [3, 5, 1, 7, 9, 6], buying when the price is 1 and selling when it's 9 gives a profit of $9 - 1 = 8$, which would be the maximum.

Approaches to Solve the Problem

1. Brute Force Approach

The most straightforward way to solve this is to try every possible buy and sell combination.

- Iterate through each day to pick a **buy day**.
- For each buy day, iterate through all subsequent days to pick a **sell day**.
- Calculate the profit for each pair (`sell_price - buy_price`).
- Keep track of the **maximum profit** found so far.

Example:

- Buy on Day 0 (price 3):
 - Sell on Day 1 (price 5): Profit $5 - 3 = 2$
 - Sell on Day 2 (price 1): Profit $1 - 3 = -2$ (invalid/loss)
 - ...and so on.
- Move to Buy on Day 1 (price 5):
 - Sell on Day 2 (price 1): Profit $1 - 5 = -4$
 - ...and so on.

Complexity Analysis:

- **Time Complexity:** $O(n^2)$ 🕒 (Due to two nested loops: one for buying, one for selling).
- **Space Complexity:** $O(1)$ (No extra space used).

2. Optimized Approach with $O(N)$ Space 🚀

We can optimize the time complexity by using a pre-processing step.

Idea: If we know the *maximum price that occurs *after* a given day*, we can easily calculate the maximum profit if we decide to buy on that day.

Method:

1. Create an **auxiliary array** (e.g., `max_right` or `max_ahead`) of the same size as the prices array.
2. Populate this array from **right to left**.
 - `max_right[i]` will store the maximum price from `prices[i]` to `prices[n-1]`.
 - Initialize `max_right[n-1] = prices[n-1]`.
 - For `i` from `n-2` down to `0`: `max_right[i] = max(prices[i], max_right[i+1])`.
3. Once `max_right` is populated, iterate through the prices array from **left to right**.
 - For each day `i`, the potential profit if buying on this day is `max_right[i] - prices[i]`.
 - Update a global `maxProfit` variable with the maximum of current `maxProfit` and the potential profit.

Example with prices = [3, 1, 4, 8, 7, 2, 5]:

Step 1: Create `max_right` array (from right to left)

- `max_right[6] = 5` (last element)
- `max_right[5] = max(prices[5]=2, max_right[6]=5) = 5`
- `max_right[4] = max(prices[4]=7, max_right[5]=5) = 7`
- `max_right[3] = max(prices[3]=8, max_right[4]=7) = 8`
- `max_right[2] = max(prices[2]=4, max_right[3]=8) = 8`
- `max_right[1] = max(prices[1]=1, max_right[2]=8) = 8`
- `max_right[0] = max(prices[0]=3, max_right[1]=8) = 8`

Resulting `max_right` array: [8, 8, 8, 8, 7, 5, 5]

Step 2: Calculate max profit (from left to right)

- Initialize `maxProfit = 0`.
- Day 0 (price 3): Profit `max_right[0]-3 = 8-3 = 5`. `maxProfit = 5`.
- Day 1 (price 1): Profit `max_right[1]-1 = 8-1 = 7`. `maxProfit = 7`.
- Day 2 (price 4): Profit `max_right[2]-4 = 8-4 = 4`. `maxProfit` remains 7.
- Day 3 (price 8): Profit `max_right[3]-8 = 8-8 = 0`. `maxProfit` remains 7.
- Day 4 (price 7): Profit `max_right[4]-7 = 7-7 = 0`. `maxProfit` remains 7.
- Day 5 (price 2): Profit `max_right[5]-2 = 5-2 = 3`. `maxProfit` remains 7.
- Day 6 (price 5): Profit `max_right[6]-5 = 5-5 = 0`. `maxProfit` remains 7.

Final `maxProfit = 7`.

Complexity Analysis:

- **Time Complexity:** $O(N)$ (One pass for `max_right`, one pass for calculating profit).
- **Space Complexity:** $O(N)$ (For the auxiliary `max_right` array).

3. Optimized Approach with $O(1)$ Space 🌟 (Ideal Solution)

The goal in interviews is often to achieve linear time complexity with constant space.

Idea: Instead of tracking the maximum price **ahead**, we track the **minimum price encountered *so far***. If we decide to sell on the current day, the best strategy is to have bought at the lowest possible price up to that point.

Method:

1. Initialize two variables:
 - `minSoFar`: Stores the minimum stock price encountered from the beginning of the array up to the current day. Initialize with the first price.
 - `maxProfit`: Stores the maximum profit found so far. Initialize with 0.
2. Iterate through the prices array from left to right (starting from the second element, or process the first element specially).
 - At each price `current_price`:
 - Update `minSoFar = min(minSoFar, current_price)`.
 - Calculate the potential profit if selling today: `potential_profit = current_price - minSoFar`.
 - Update `maxProfit = max(maxProfit, potential_profit)`.
3. After iterating through all prices, `maxProfit` will hold the maximum possible profit.

Algorithm Logic: As we iterate through the prices, we always assume we are **selling** on the current day. To maximize profit when selling on the current day, we must have bought at the lowest price seen **before or on** that day. By continuously updating `'minSoFar'`, we ensure that `'current_price - minSoFar'` gives us the best profit achievable if we sell on `'current_price'` day.

Example Dry Run with prices = [5, 2, 6, 1, 4]:

- Initialize `maxProfit = 0`.
- Initialize `minSoFar = prices[0] = 5`.

-
- **Day 0 (price = 5):**

- $\text{minSoFar} = \min(5, 5) = 5$
 - $\text{profit} = 5 - 5 = 0$
 - $\text{maxProfit} = \max(0, 0) = 0$
 - **Day 1 (price = 2):**
 - $\text{minSoFar} = \min(5, 2) = 2$
 - $\text{profit} = 2 - 2 = 0$
 - $\text{maxProfit} = \max(0, 0) = 0$
 - **Day 2 (price = 6):**
 - $\text{minSoFar} = \min(2, 6) = 2$ (minSoFar remains 2)
 - $\text{profit} = 6 - 2 = 4$
 - $\text{maxProfit} = \max(0, 4) = 4$
 - **Day 3 (price = 1):**
 - $\text{minSoFar} = \min(2, 1) = 1$
 - $\text{profit} = 1 - 1 = 0$
 - $\text{maxProfit} = \max(4, 0) = 4$ (maxProfit remains 4)
 - **Day 4 (price = 4):**
 - $\text{minSoFar} = \min(1, 4) = 1$ (minSoFar remains 1)
 - $\text{profit} = 4 - 1 = 3$
 - $\text{maxProfit} = \max(4, 3) = 4$ (maxProfit remains 4)
-

Final Result: The function returns $\text{maxProfit} = 4$. This profit is achieved by buying on Day 1 (price 2) and selling on Day 2 (price 6).

Complexity Analysis:

- **Time Complexity:** $O(N)$ (Single pass through the array).
- **Space Complexity:** $O(1)$ (Only two variables are used).

Conclusion

The **$O(N)$ time and $O(1)$ space** solution is considered the ideal and most efficient way to solve the "Stock Buy and Sell, Variant 1" problem. It's a fundamental problem that highlights dynamic programming ideas and efficient variable tracking.

Stay tuned for the next video where we will explore variants allowing multiple stock transactions! 🙌